Machine Learning - TP2

Projet 2 : Les réseaux de neurones

Vincent Tourenne & Lison Boo



Table des Matières

I - Algorithme du perceptron simple..... 2
II - Apprentissage de Widrow Hoff.....3
1. Principe et traitement par lots..... 3
2. Vérification sur le OU logique..... 4
3. Ensemble de Test 1 : Données linéairement séparables..... 5
Apprentissage par ligne contre apprentissage par lot.....5
Autres tests avec des initialisations différentes..... 6
4. Ensemble de Test 2 : Limites de la frontière linéaire.....7
III - Perceptron multicouche..... 8
IV - Classification d'images à partir des descripteurs..... 9
1. Chargement des descripteurs.....9
2. Développement du réseau dense..... 11
Développement du perceptron.....11
Choix des mesures : PHOG vs JCD vs CEDD vs FCTH vs Fuzzy..... 12
1. Résultats avec PHOG..... 13
2. Résultats avec JCD.....14
3. Résultats avec CEDD.....15
4. Résultats avec FCTH.....16
5. Résultats avec Fuzzy Color Histogram..... 17
Choix des mesures : Concaténation..... 18
Paramétrage de l'algorithme : choix des hyperparamètres.....20
1. Nombre d'époques.....20
2. Nombre de neurones..... 21
3. Nombre de couches cachées..... 22
4. Activation des couches cachées.....23
5. Réseau final..... 24
Comparaison avec l'algorithme KppV.....25
1. Réalisation de l'algorithme KppV..... 25
2. Résultats et comparaison..... 28
V - Classification d'images par réseau profond : apprentissage des descripteurs..... 29
Architectures utilisées..... 29

I - Algorithme du perceptron simple

On représente un neurone par son potentiel `v = w0 + w1 * x1 + w2 * x2` où w est le vecteur de poids et x est le vecteur d'entrées. On peut coder une fonction assez simple qui calcule ce potentiel, puis applique `sign` ou `tanh` selon la valeur du paramètre `active`.

```
def perceptron_simple(x, w, active):
    result = w[0] + x[0]*w[1] + x[1]*w[2]

    if (active == 0):
        return np.sign(result)
    elif (active==1):
        return np.tanh(result)
    return;
```

Extrait 1 - Calcul du potentiel et de l'activation du neurone

On peut ensuite tester cette nouvelle méthode avec l'exemple du OU logique avec les poids demandés w = [-0,5 ; 1 ; 1]. Les réponses attendues valent -1 pour l'entrée (0, 0), et +1 pour les autres.

Entrée (x1, x2)	Potentiel v	Sortie sign(v)	Sortie attendue
(0, 0)	-0,5	-1	-1
(0, 1)	0,5	+1	+1
(1, 0)	0,5	+1	+1
(1, 1)	1,5	+1	+1

Tableau 1 - Fonction OU

La frontière de décision vérifie v = 0. Le point (0, 0) se trouve du côté négatif de cette droite, et les trois autres du côté positif.

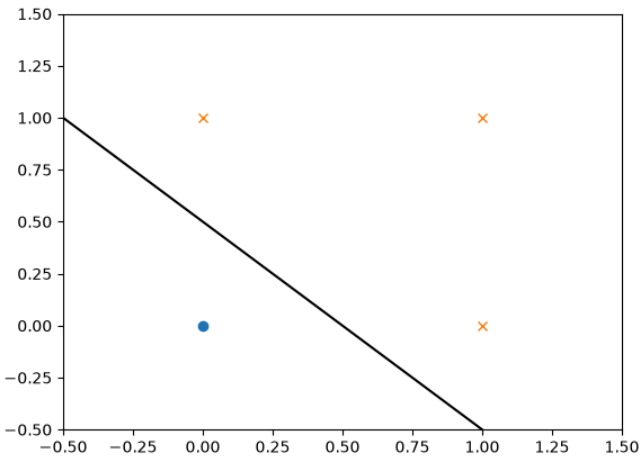


Figure 1 - Résultat du perceptron : graphe individu/frontière

II - Apprentissage de Widrow Hoff

1. Principe et traitement par lots

On utilise tanh pendant l'apprentissage, car cette activation est dérivable. Pour un individu i, la correction des poids correspond à la formule :

$$\Delta w = \alpha\,(yd_i - y_i)\,(1 - y_i^2)\,\tilde{x}_i$$

Les poids initiaux sont tirés aléatoirement entre -0,5 et 0,5. A chaque époque, les indices des individus sont mélangés. Les sorties d'un lot sont calculées avec les mêmes poids, ses corrections sont ensuite additionnées avant une unique mise à jour.

```
for debut in range(0, yd.size, batch_size):
    lot = indices[debut:debut + batch_size]

    # tableau des résultats du lot
    y_result = np.zeros(len(lot))

    # remplir les résultats avec les poids actuels
    for j in range(len(lot)):
        indice = lot[j]
        y_result[j] = perceptron_simple(x[:, indice], w, 1)

    # somme des corrections du lot
    delta_w = np.zeros(3)

    for j in range(len(lot)):
        indice = lot[j]

        # calcul de la correction
        xi = np.array([1, x[0, indice], x[1, indice]])
        delta_w += alpha*(yd[indice]-y_result[j])*(1-y_result[j]**2)*xi

        # cumul de l'erreur de cette époque
        erreur[i] += (yd[indice]-y_result[j])**2

    # ajustement des poids après le lot
    w += delta_w
```

Extrait 2 - Somme des corrections et mise à jour de chaque lot

Avec `batch_size = 1`, les poids sont actualisés après chaque individu (**apprentissage en ligne**). Avec `batch_size = N`, ils ne sont plus actualisés qu'une seule fois par époque (**apprentissage par lot**).

A la fin de chaque époque, on compte les erreurs de classification avec sign($w^T \tilde{x}$). L'apprentissage s'arrête lorsque ce nombre est nul, ou lorsque le nombre maximal d'époques est atteint.

2. Vérification sur le OU logique

Pour tester notre algorithme, on peut tester son apprentissage sur l'exemple précédent du OU logique.

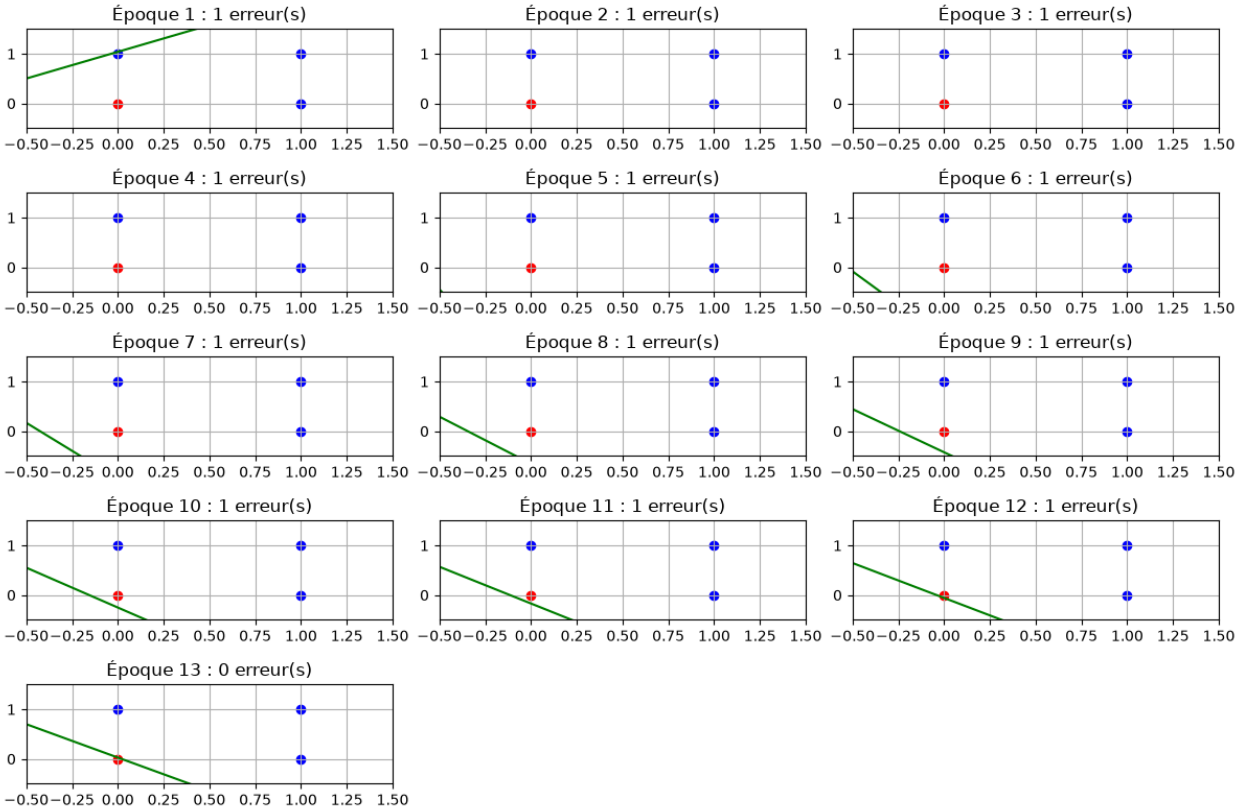


Figure 2 - Apprentissage du OU logique

Le programme s'arrête de lui-même à la treizième époque, les poids obtenus sont environ [-0,01468 ; 0,73633 ; 0,54676] et les réponses finales sont bien [-1, +1, +1, +1]. L'apprentissage trouve donc bien une frontière qui sépare correctement les quatre entrées !

3. Ensemble de Test 1 : Données linéairement séparables

Apprentissage par ligne contre apprentissage par lot

L'ensemble d'apprentissage p2_d1.txt présente 50 individus distincts équitablement scindés en deux classes de 25. On teste notre algorithme en comparant des lots de taille 1 (apprentissage par ligne) contre des lots de taille 50 (apprentissage par lot)

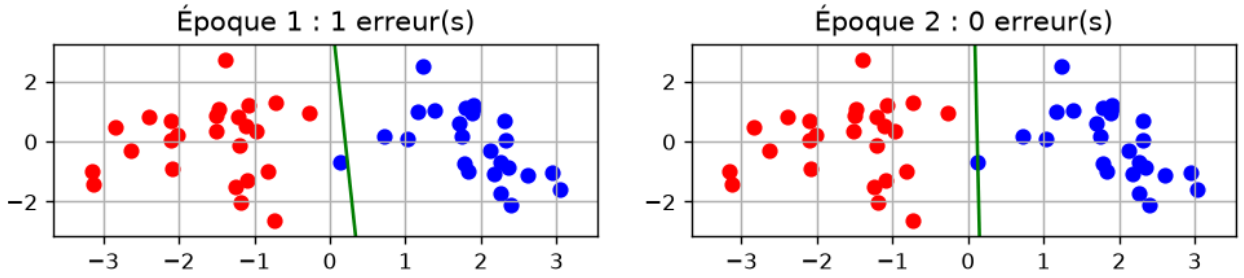


Figure 3 - Test 1 en ligne - Poids finaux : [-0,14 ; 1,13 ; 0,01]

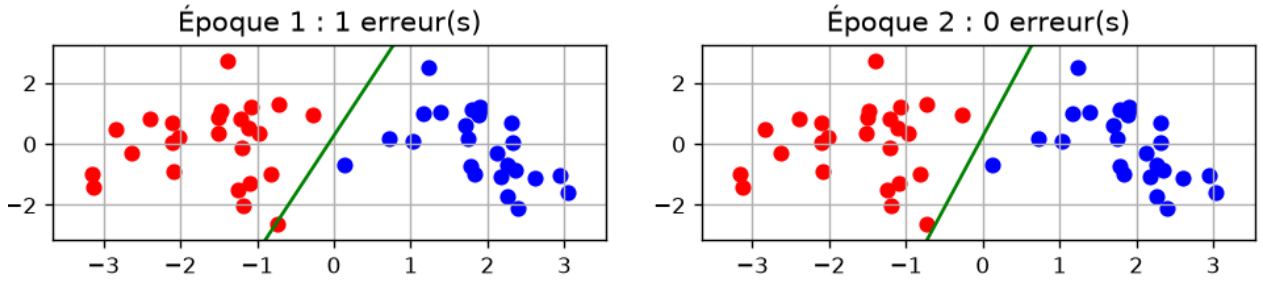


Figure 4 - Test 1 en lot - Poids finaux : [0,3 ; 6,9 ; -1,5]

Les deux modes trouvent une frontière séparant les classes en seulement deux époques dans ces essais. Les poids finaux diffèrent du fait de la variation initiale des premiers poids définis dans l'algorithme.

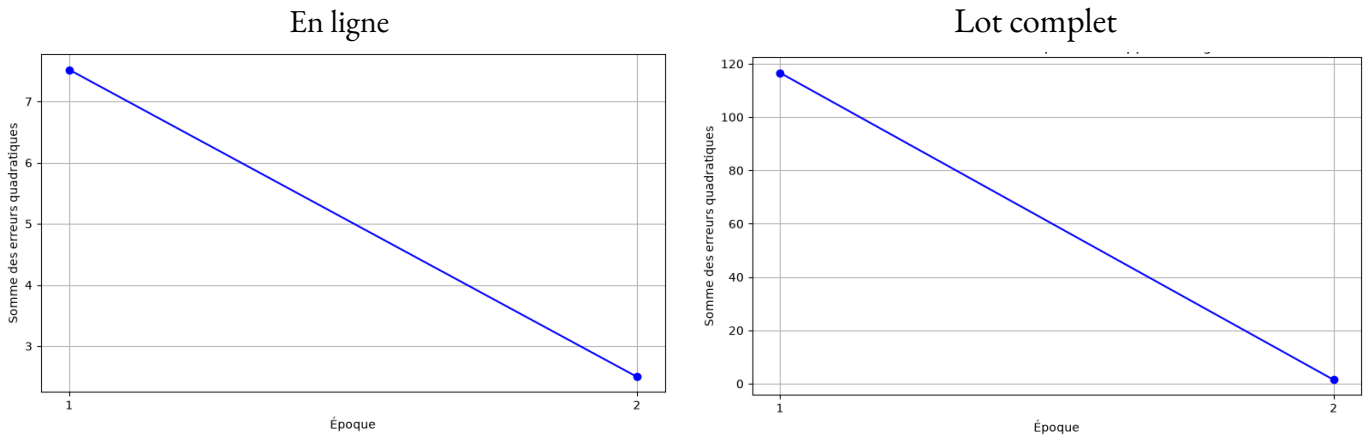


Figure 5 - Erreur quadratique cumulée par époque

Autres tests avec des initialisations différentes

Pour pousser plus loin, on peut tester notre algorithme plusieurs fois afin de voir ce qui se passe quand il choisit des poids initiaux très différents :

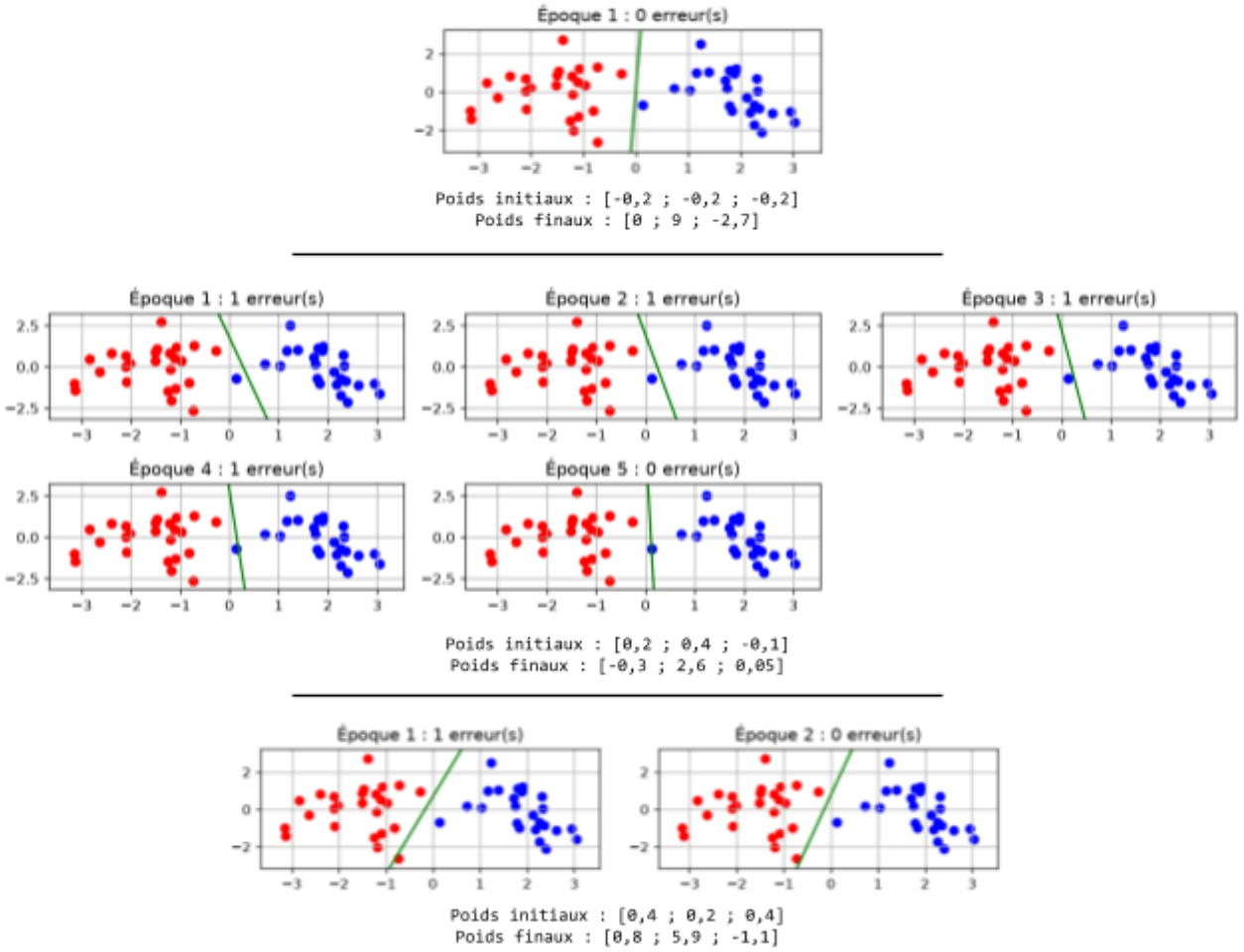


Figure 6 - Trois initialisations réalisées sur Test 1

Sur 10 000 itérations testées, le nombre moyen d'époques est d'environ 2,20.

En conclusion, on peut estimer l'ensemble Test 1 comme étant adapté à un perceptron simple, car les deux classes peuvent être séparées simplement par une unique droite. L'initialisation influence le chemin suivi et la frontière retenue.

4. Ensemble de Test 2 : Limites de la frontière linéaire

L'ensemble Test 2 conserve le même format et les mêmes étiquettes, mais présente des données bien plus bruitées : les deux classes ne sont pas si facilement séparables. On obtient ainsi, avec notre perceptron simple, en apprentissage par ligne :

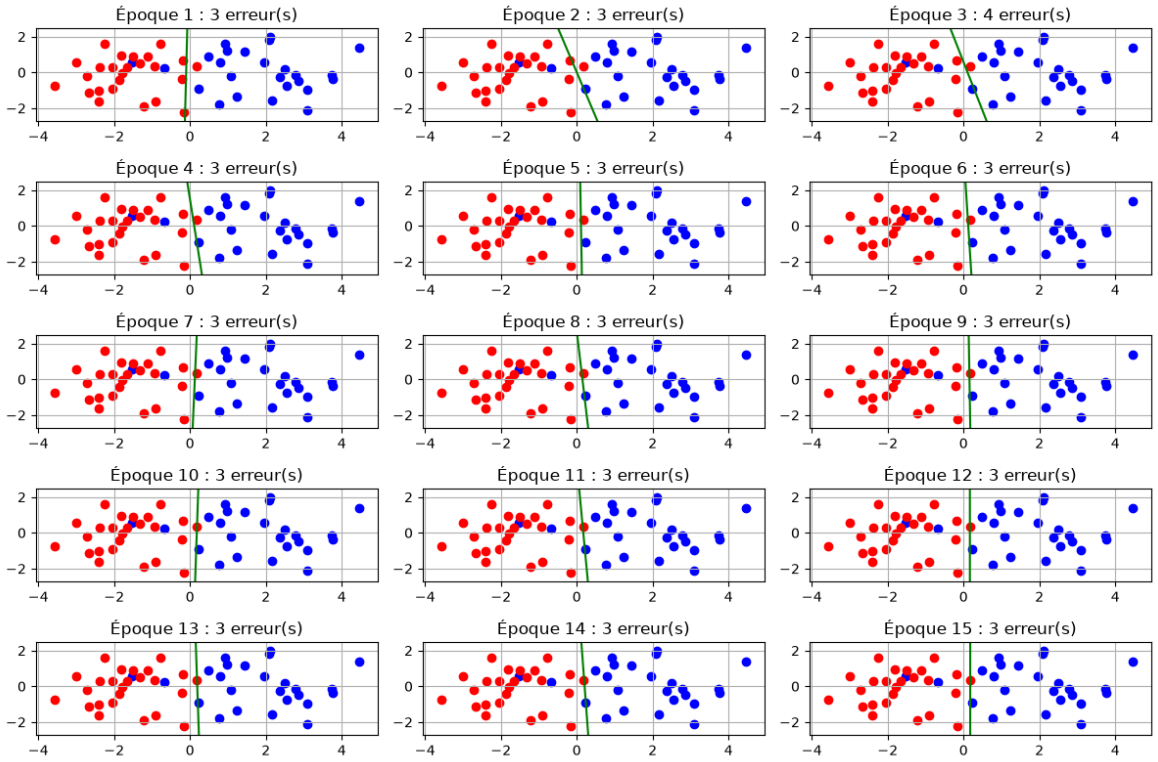


Figure 7 - Résultat du perceptron sur Test 2 en apprentissage par ligne

La limite de quinze époques est atteinte sans classification complète. La frontière évolue, mais elle ne sépare pas correctement tous les individus. On observe assez facilement le problème : les données ne sont pas séparables par une simple droite, un simple perceptron.

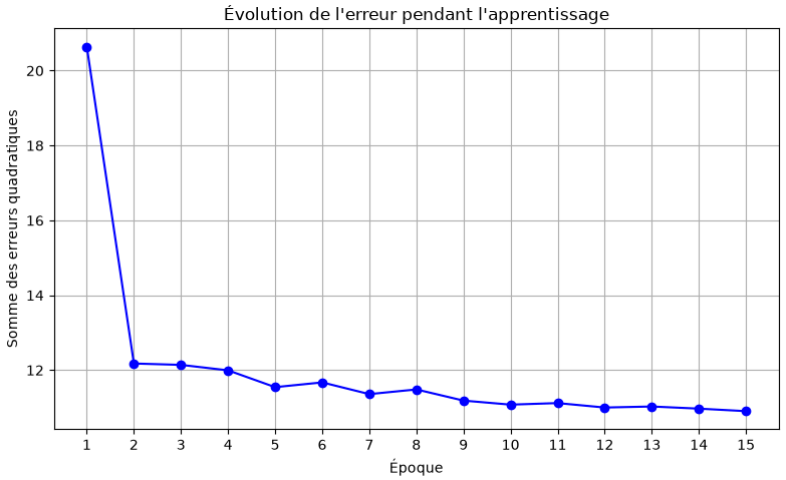


Figure 8 - Erreur quadratique sur Test 2 pendant les 15 époques

III - Perceptron multicouche

Le perceptron multicouche emploie une stratégie similaire à celle du perceptron classique. Elle est simplement étendue à plusieurs couches de neurones : sur la couche 1, chacun possède ses poids et sorties, qui sont ensuite branchées sur la couche supérieure pour une sortie finale. On peut ainsi coder cela très simplement, en utilisant ici la fonction d'activation sigmoïde :

```
def sigmoide(v):
    return 1/(1+math.exp(-1*v));

def multiperceptron(x, w1, w2):
    # calcul de la sortie du premier neurone
    v1 = w1[0][0] + w1[1][0]*x[0] + w1[2][0]*x[1]
    y1 = sigmoide(v1)

    # calcul de la sortie du deuxième neurone
    v2 = w1[0][1] + w1[1][1]*x[0] + w1[2][1]*x[1]
    y2 = sigmoide(v2)

    # calcul de la sortie du perceptron
    v3 = w2[0] + w2[1]*y1 + w2[2]*y2
    y3 = sigmoide(v3)

    return y3
```

Extrait 3 - Propagation de l'entrée jusqu'à la sortie du perceptron multicouche

Avec cette méthode, on peut lancer un simple test avec les valeurs suivantes :

- x = [1, 1], c'est l'entrée testée,
- w1 = [[-0.5, 0.5], [2,1], [-1, 0.5]] : les poids des deux neurones ({-0.5, 2, -1} et {0.5, 1, 0.5}),
- w2 = [2,-1,1], le neurone de sortie.

On obtient ainsi en sortie la valeur : `0.905367309540257`. Cela est assez proche du calcul manuel :

```
v1 = w1[0][0] + w1[1][0]*x[0] + w1[2][0]*x[1]
v1 = -0.5 + 2*1 + (-1)*1
v1 = 0.5

v2 = 0.5 + 1*1 + 0.5 = 2

y1 = 1/(1+exp(-0.5)) = 0.62
y2 = 1/(1+exp(-2)) = 0.88

v3 = 2 + (-1)*0.62 + 1*0.88 = 2.26
y3 = 1/(1+exp(-2.26)) = 0.905
```

Figure 9 - Calcul manuel du test réalisé

IV - Classification d'images à partir des descripteurs

1. Chargement des descripteurs

Nous utilisons la base Wang, composée de 1000 images réparties en dix classes de 100 images. Chaque image est représentée par cinq descripteurs fournis dans le fichier `WangSignatures.xlsx` : PHOG, JCD, CEDD, FCTH et Fuzzy Color Histogram.

Chaque feuille est tout d'abord importée dans un DataFrame avec pandas. La première colonne contient les noms des images, les autres colonnes constituent les vecteurs de mesures. Nous obtenons ainsi cinq tableaux représentant les mêmes images, avec un nombre de caractéristiques propre à chaque descripteur.

Les étiquettes sont calculées à partir des noms des fichiers : l'image i.jpg appartient à la classe correspondant à la division entière de i par 100.

```
filepath = "TP2/.ressources/WangSignatures.xlsx" # Lien vers le fichier Excel

# On met le fichier dans des dataframes
df_PHOG = pd.read_excel(filepath, sheet_name = "WangSignaturesPHOG",
header=None, index_col=0)
df_JCD = pd.read_excel(filepath, sheet_name = "WangSignaturesJCD",
header=None, index_col=0 )
df_CEDD = pd.read_excel(filepath, sheet_name = "WangSignaturesCEDD",
header=None, index_col=0 )
df_FCTH = pd.read_excel(filepath, sheet_name = "WangSignaturesFCTH",
header=None, index_col=0 )
df_Fuzzy = pd.read_excel(filepath, sheet_name =
"WangSignaturesFuzzyColorHistogr", header=None, index_col=0 )

# Vecteurs de données
X_PHOG = df_PHOG.values.astype(float) # mesures (1000 x d)
X_JCD = df_JCD.values.astype(float)
X_CEDD = df_CEDD.values.astype(float)
X_FCTH = df_FCTH.values.astype(float)
X_Fuzzy = df_Fuzzy.values.astype(float)
# Vecteur de labels (identiques pour chaque jeu de données)
label = np.array([int(nom[:-4]) // 100 for nom in df_PHOG.index])
```

Extrait 4 - Code pour construire les 5 tableaux de mesures et le vecteur label pour assigner la classe pour chaque ligne

Les données sont ensuite séparées lors de l'entraînement :

```
def train_model(X, label, name):
    # On fait une séparation 80/20 des données entre apprentissage et test
    X_train, X_test, label_train, label_test = train_test_split(
        X,
        label,
        test_size=0.2,
        random_state=42, # seed d'aléatoire
        stratify=label
    )
```

Extrait 5 - Code de séparation des données pour l'entraînement

Nous réservons aléatoirement 20 % des images au test, soit 200 images. Le paramètre `stratify` conserve les proportions des dix classes dans cette séparation. Le paramètre `random_state` fixe le découpage afin de retrouver les mêmes images de test lors des différentes comparaisons.

Parmi les 800 images restantes, Keras réserve 20 % à la validation avec `validation_split=0.2` (cf suite de la méthode train_model). Cet ensemble permet de suivre les performances durant l'entraînement sans participer à la mise à jour des poids. L'apprentissage effectif utilise donc 640 images, la validation 160 images et le test 200 images.

2. Développement du réseau dense

Développement du perceptron

Nous utilisons un perceptron multicouche réalisé avec Keras. Son entrée correspond au **vecteur de mesures** de l'image, sa dimension dépend donc du descripteur utilisé.

Le réseau possède **deux couches cachées** de 16 et 8 neurones, utilisant l'activation **ReLU**. La couche de sortie comporte dix neurones avec une activation **softmax**. L'entraînement suit **vingt époques**.

Nous suivons **l'accuracy**, qui mesure la **proportion d'images correctement classées**, et la loss, qui tient compte des probabilités attribuées aux classes. Le **taux d'erreur** correspond à `1-accuracy`.

```
from tensorflow import keras
num_classes = 10
input_space = X_train.shape[1]  # taille des données

model = keras.models.Sequential([
    keras.layers.Dense(16, activation="relu", input_shape=(input_space,)),
    keras.layers.Dense(8, activation='relu'),
    keras.layers.Dense(num_classes, activation='softmax')
])

model.compile(loss="categorical_crossentropy",
              optimizer="adam",
              metrics=["accuracy"])

from tensorflow.keras.utils import to_categorical

history = model.fit(
    X_train,
    to_categorical(label_train),
    epochs=20, # 20 époques
    batch_size=32, # 32 images d'entraînement par mise à jour des poids
    validation_split=0.2
)

score = model.evaluate(X_test, to_categorical(label_test))
```

Extrait 6 - Suite de la fonction "train_model" pour l'entraînement des données

Choix des mesures : PHOG vs JCD vs CEDD vs FCTH vs Fuzzy

Nous comparons les cinq descripteurs en conservant la même architecture et les mêmes paramètres d'entraînement. Les résultats suivants sont mesurés sur les 200 images de test.

Descripteur	**Accuracy sur le test**	**Taux d'erreur**	**Loss sur le test**
PHOG	34,50%	65,50%	1,8649
JCD	73,00%	27,00%	0,7319
CEDD	73,00%	27,00%	0,8063
FCTH	58,00%	42,00%	1,2337
Fuzzy	23,50%	76,50%	2,0765

Tableau 2 - Classification avec chaque descripteur

Dans cette expérience, **JCD et CEDD** obtiennent la meilleure accuracy, suivis de FCTH. PHOG et Fuzzy donnent des résultats plus faibles avec les conditions d'apprentissage retenues. Les pages suivantes précisent ces différences à l'aide des courbes et matrices de confusion.

1. Résultats avec PHOG

L'accuracy et les pertes progressent encore à 20 époques, mais le test n'atteint que 34,5% soit 69 bonnes classifications. Les images de classe 3 sont reconnues à 17/20, tandis qu'aucune image des classes 0, 5 et 7 n'est correctement classée. Cette configuration distingue certaines classes et échoue sur d'autres.



Figure 10 - Accuracy et loss avec PHOG

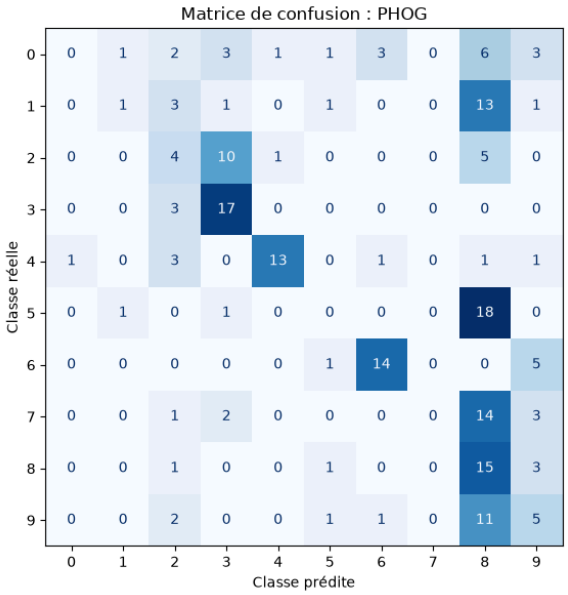


Figure 11 - Matrice de confusion avec PHOG

2. Résultats avec JCD

JCD classe correctement 146 images sur 200, notamment celles des classes 4 et 6 qui sont parfaitement reconnues. Les pertes diminuent sur les deux ensembles, avec un écart qui se creuse entre apprentissage et validation.

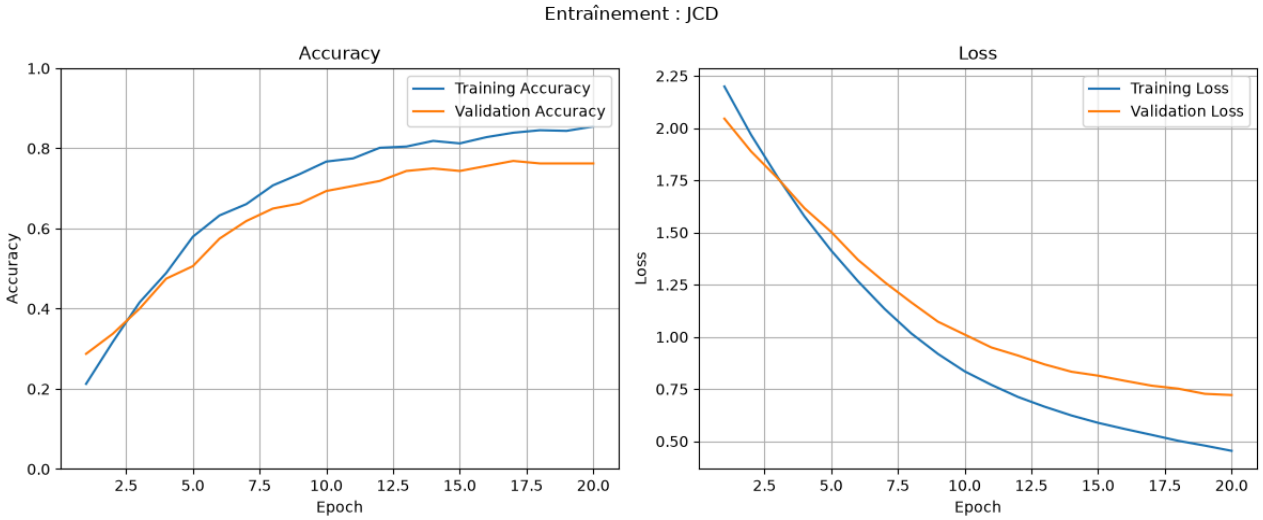


Figure 12 - Accuracy et loss avec JCD

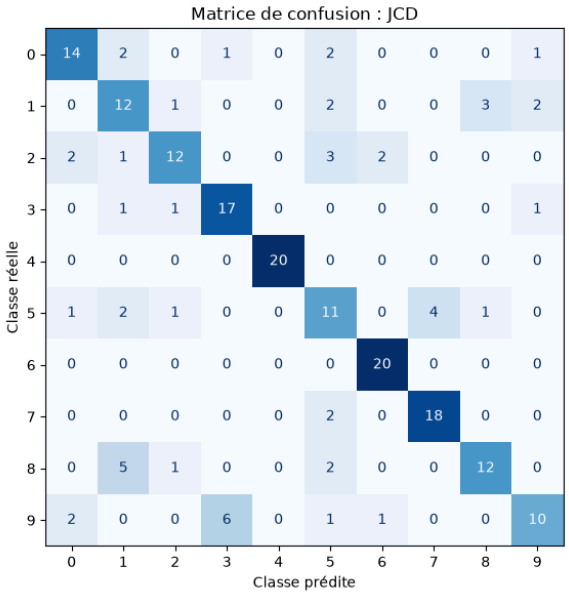


Figure 13 - Matrice de confusion avec JCD

3. Résultats avec CEDD

CEDD atteint également 146 bonnes classifications, avec tout de même des différences visibles sur sa matrice de confusion avec JCD (une plus mauvaise reconnaissance de la classe 0, par exemple).

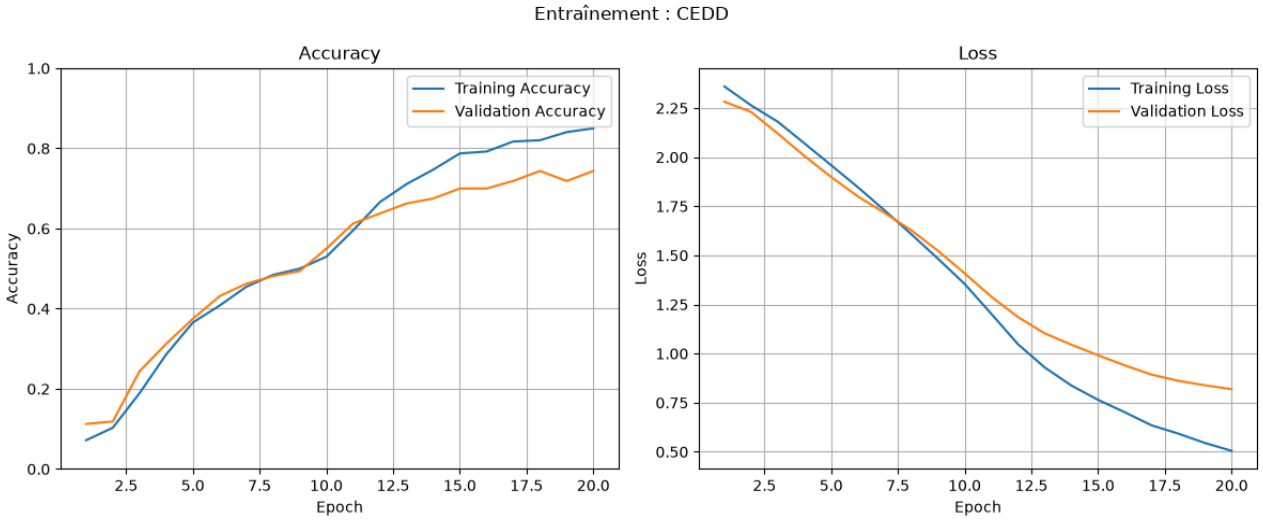


Figure 14 - Accuracy et loss avec CEDD

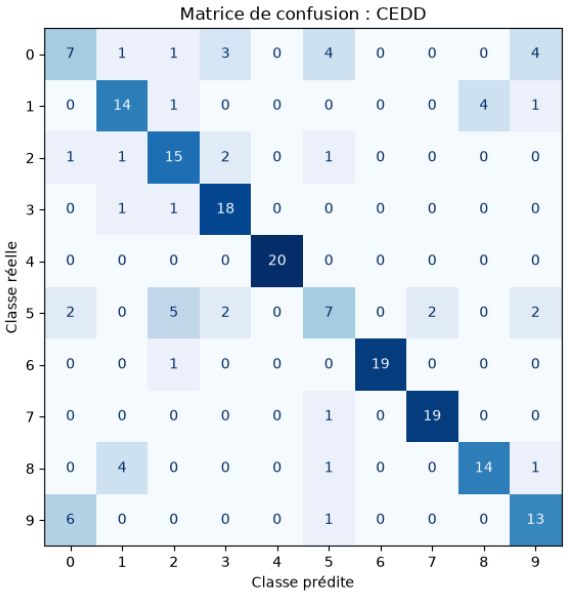


Figure 15 - Matrice de confusion avec CEDD

4. Résultats avec FCTH

FCTH atteint 116 bonnes classifications, notamment les classes 4, 6 et 7, alors que les classes 0, 2 et 5 restent difficiles. Les courbes continuent globalement à diminuer sur les deux ensembles.

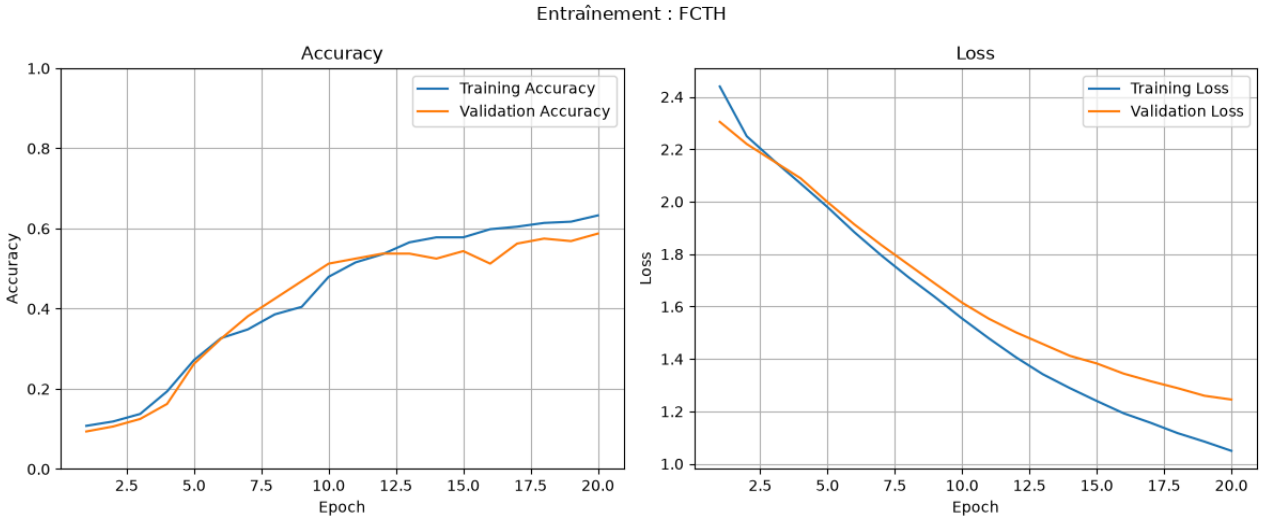


Figure 16 - Accuracy et loss avec FCTH

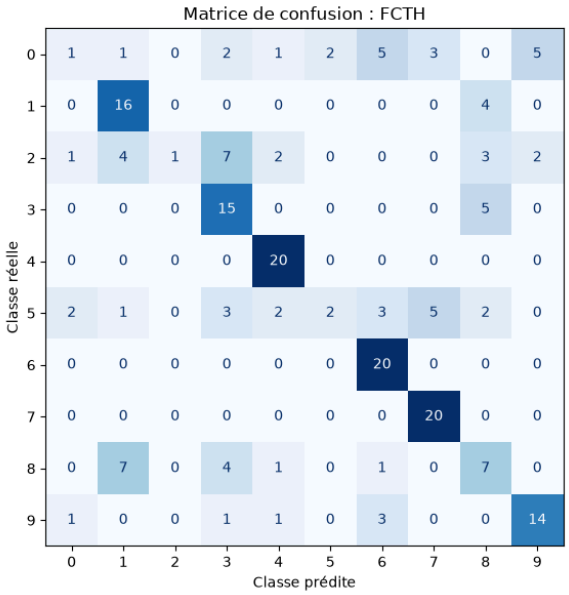


Figure 17 - Matrice de confusion avec FCTH

5. Résultats avec Fuzzy Color Histogram

Avec Fuzzy, les scores d'accuracy restent faibles et stagnent rapidement. La matrice montre une forte concentration sur la classe 8 : 166 images y sont placées sur les 200 d'entrée. Seule anomalie : les 20 images de classe 4 qui sont parfaitement reconnues.

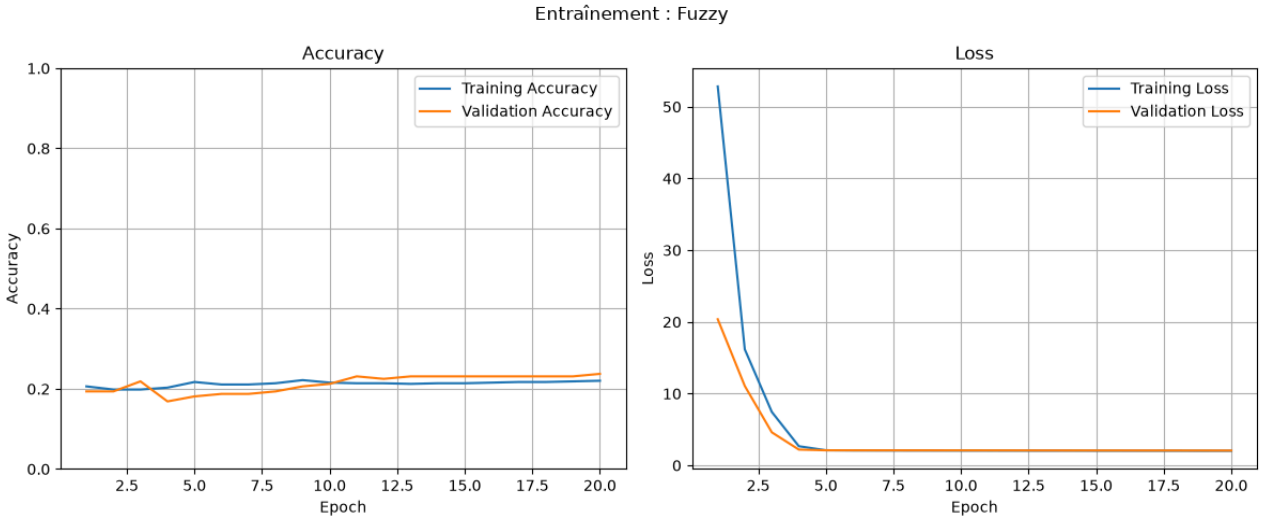


Figure 18 - Accuracy et loss avec Fuzzy

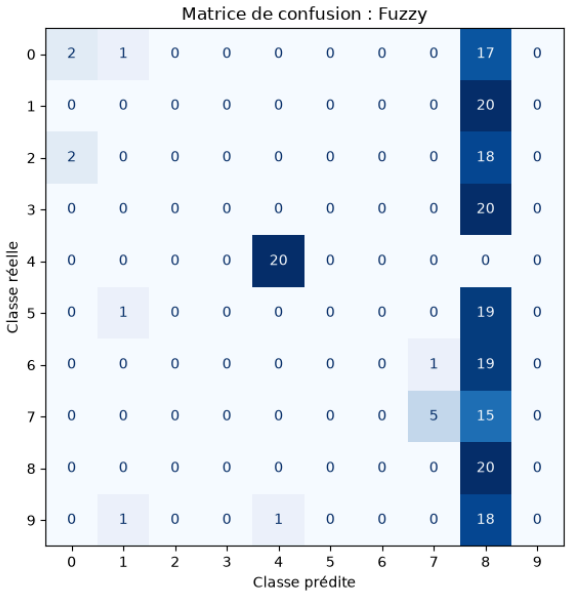


Figure 19 - Matrice de confusion avec Fuzzy

Choix des mesures : Concaténation

On peut ensuite concaténer nos cinq descripteurs pour voir le résultat :

Ensemble de données	Accuracy sur le test	Taux d'erreur	Loss sur le test
Concaténées (toutes)	20,00%	80,00%	2,0666

Tableau 3 - Résultat de la concaténation complète

Les résultats ne donnent que 40 bonnes classifications. Les courbes stagnent rapidement et la matrice montre 177 prédictions de la classe 6. Ici encore, les 20 images de la classe 4 sont cependant parfaitement reconnues.

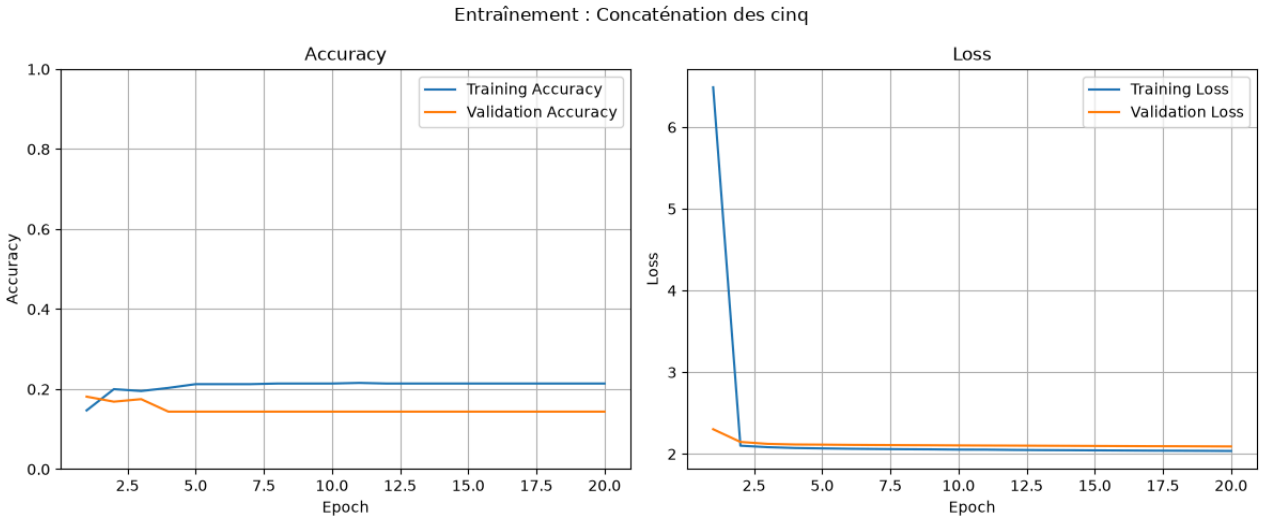


Figure 20 - Accuracy et loss avec concaténation complète des cinq descripteurs

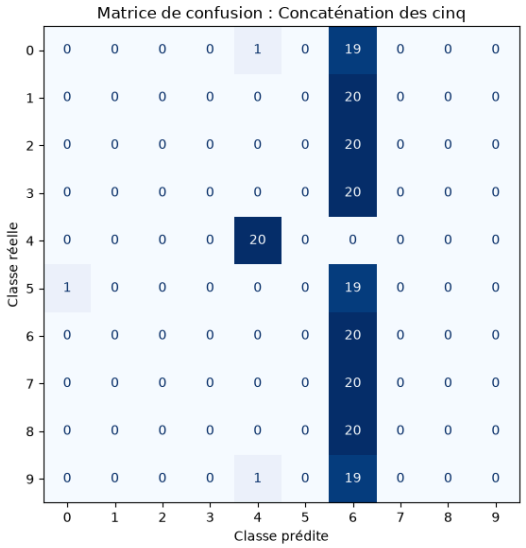


Figure 21 - Matrice de confusion avec concaténation complète des cinq descripteurs

Ajouter toutes les mesures ne suffit donc pas à améliorer cette configuration.

On peut alors plutôt tenter de ne concaténer que les mesures les plus performantes dans ces premiers essais : JCD, CEDD et FCTH :

Ensemble de données	Accuracy	Taux d'erreur	Loss
Concaténées (JCD, CEDD, FCTH)	79,00%	21,00%	0,6020

Tableau 4 - Résultats de la concaténation retenue

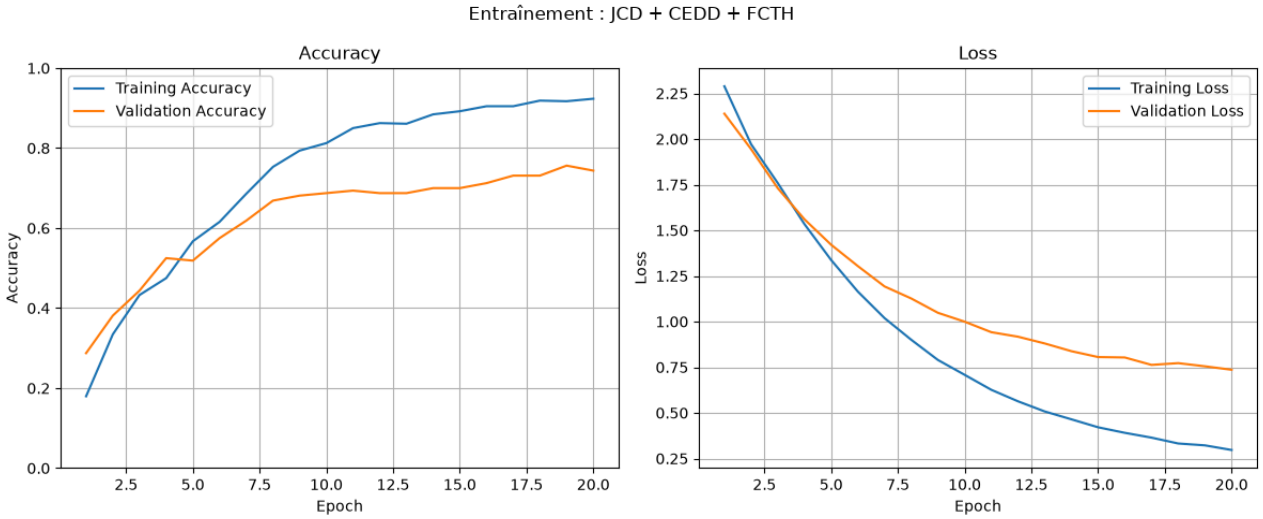


Figure 22 - Accuracy et loss avec concaténation avec JCD CEDD FCTH

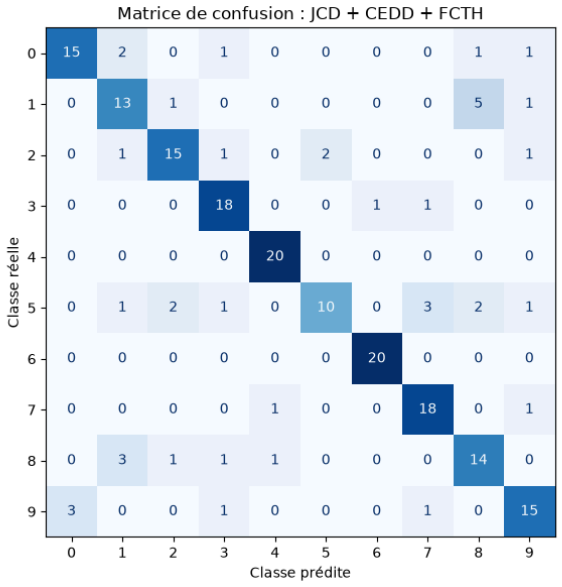


Figure 23 - Matrice de confusion avec concaténation avec JCD CEDD FCTH

La concaténation de JCD, CEDD et FCTH atteint une accuracy de 79 %, soit un taux d'erreur de 21 %. Elle dépasse la concaténation complète, mais également JCD et CEDD utilisés séparément : le gain est de 6 points d'accuracy, soit 12 images supplémentaires correctement classées sur les 200 images de test.

Paramétrage de l'algorithme : choix des hyperparamètres

On peut faire varier les hyperparamètres sur plusieurs axes en utilisant la concaténation de JCD, CEDD et FCTH :

1. **Nombre d'époques**

Par défaut, le nombre d'époques choisies (le nombre de cycles d'entraînement du modèle) est de 20. On peut essayer de voir si augmenter celui-ci nous donne un plus faible taux d'erreur :

Configuration	Accuracy	Taux d'erreur	Loss
20 époques	76,00 %	24,00 %	0,7757
50 époques	78,00 %	22,00 %	0,5805
100 époques	82,50 %	17,50 %	0,6912
150 époques	83,00 %	17,00 %	0,8953

Tableau 5 - Effet du nombre d'époques

Passer de 20 à 50 époques peut être profitable, avec un taux d'erreur et une mesure de perte légèrement réduits. Passer à 100 époques réduit encore le taux d'erreur de 4,5%, mais on observe une **augmentation de la perte**. Cette observation se poursuit encore à 150 époques, avec une diminution du taux d'erreur de 0.5% pour une autre augmentation de la perte.

Le comportement de cette perte peut signaler un problème sous-jacent de **surentraînement**. Il est possible de l'étudier sur un graphe d'entraînement :

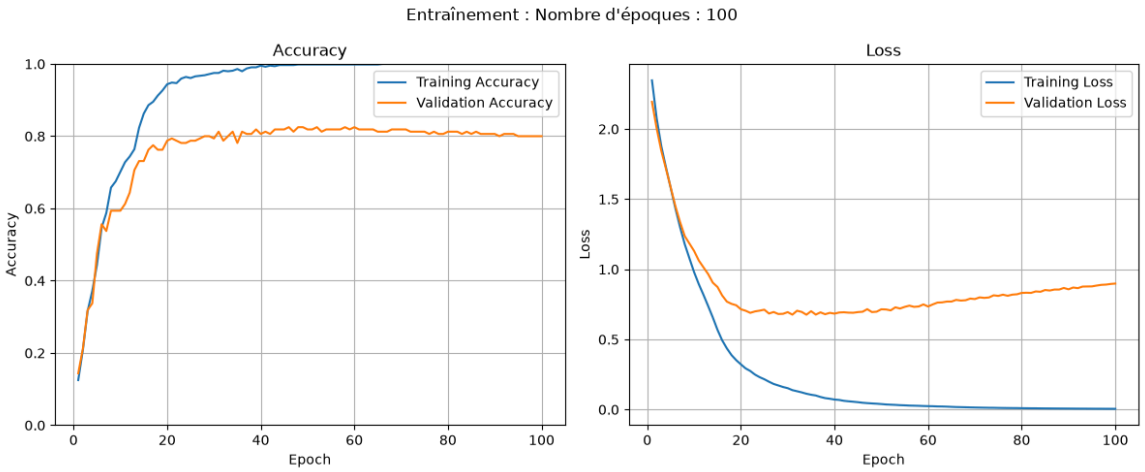


Figure 24 - Entraînement prolongé jusqu'à 100 époques

On observe deux phénomènes :

- **l'accuracy** sur les ensembles d'entraînement **plafonne à 100%** dès la quarantième époque,

- la **mesure de perte** sur les ensembles de validation stagne de l'époque 20 à 50, puis **augmente**.

Ces deux phénomènes nous le confirment : notre réseau est en **surentraînement** au-delà d'une quarantaine d'époques.

2. Nombre de neurones

Le réseau que nous avons entraîné possède deux couches, à 16 et 8 neurones. Le nombre de neurones par couche est quelque chose que l'on peut faire varier :

Neurones des couches cachées	Accuracy	Taux d'erreur	Loss
(8, 4)	52,50 %	47,50 %	1,4934
(16, 8)	83,50 %	16,50 %	0,5801
(32, 16)	85,00 %	15,00 %	0,4774
(64, 32)	87,00 %	13,00 %	0,4588
(128, 64)	86,50 %	13,50 %	0,4923

Tableau 6 - Effet du nombre de neurones dans les deux couches cachées

On observe, assez logiquement, que diminuer ce nombre de neurones réduit amplement la précision de notre classification.

A l'inverse, doubler, puis quadrupler notre nombre de neurones nous indique un taux d'erreur plus faible, bien que les améliorations soient moins drastiques comparativement à l'investissement en neurones. On gagne 2,5% d'accuracy en doublant les neurones, puis seulement 2% en les quadruplant.

Enfin, pousser encore plus loin avec un nombre de neurones multiplié par 8 expose à nouveau un problème de surapprentissage, avec une perte qui augmente et un graphe d'entraînement qui plafonne.

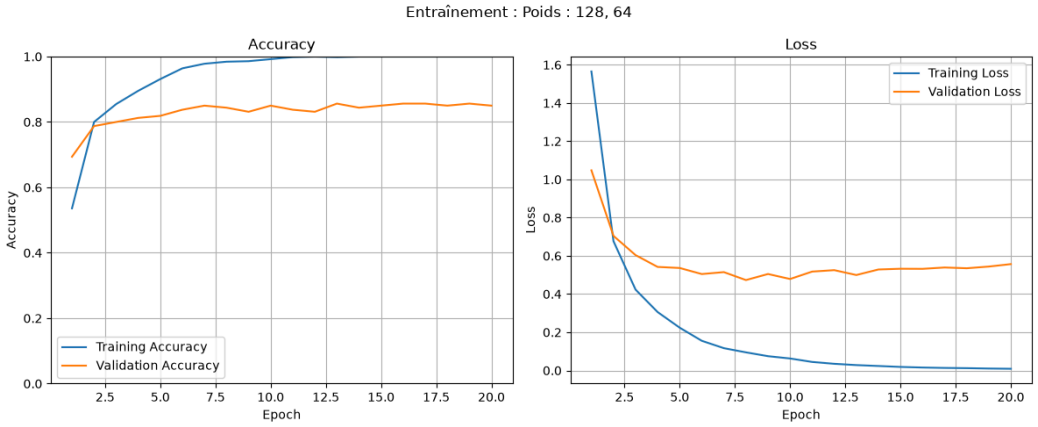


Figure 25 - Courbes du réseau à 128 et 64 neurones

On peut donc cibler assez raisonnablement un nombre de neurones [16, 8], voire élargir à [32, 16] pour notre réseau.

3. Nombre de couches cachées

Notre réseau possède deux couches cachées. Il nous est possible de faire varier ce nombre indépendamment du nombre de neurones par couche.

Configuration	Accuracy	Taux d'erreur	Loss
1 couche (16)	85,00 %	15,00 %	0,4795
2 couches (16, 8)	77,00 %	23,00 %	0,6753
3 couches (16, 8, 8)	71,50 %	28,50 %	0,8628

Tableau 7 - Effet du nombre de couches cachées

Contrairement à notre intuition initiale, on se rend compte ici que **l'augmentation du nombre de couches cachées entraîne ici une diminution de l'accuracy de notre réseau**, passant de 85% avec une seule couche à 71,5% avec trois. Cette dégradation est accompagnée d'une augmentation de la perte, de 0,4795 à 0,8628.

L'étude des graphiques permet de constater que les trois configurations continuent globalement à apprendre, avec une diminution des pertes d'entraînement et de validation. Le modèle à trois couches présente cependant un écart plus important entre les performances d'entraînement et de validation, ce qui peut signifier une moins bonne généralisation.

Ainsi, on ne peut **pas** attribuer cette dégradation à un surapprentissage.

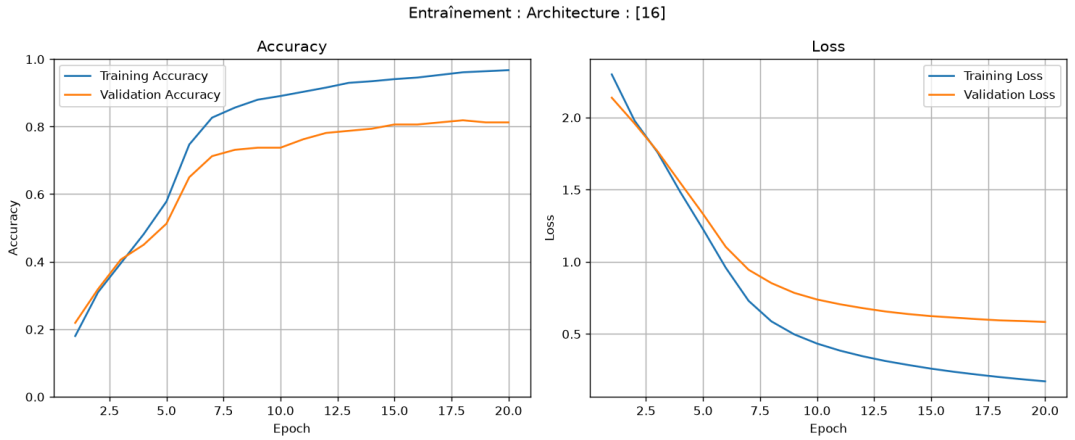


Figure 26 - Courbes avec une couche cachée de 16 neurones

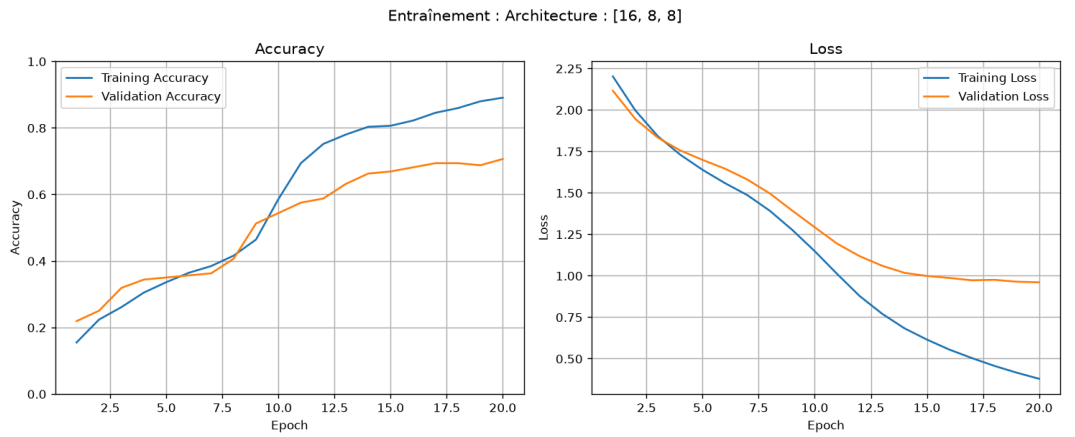


Figure 27 - Courbes avec trois couches cachées de 16, 8 et 8 neurones

Une autre explication possible est que la **complexité du modèle** devient trop importante par rapport à la taille de notre jeu de données.

Dans tous les cas, dans notre expérience, une architecture plus simple avec une seule couche cachée à 16 neurones semble donc mieux adaptée. Approfondir le réseau apporte peu.

4. Activation des couches cachées

Enfin, le dernier hyperparamètre sur lequel nous avons le contrôle est la **fonction d'activation des couches cachées**. Le modèle actuel utilise ReLU, mais nous pouvons aussi tester tanh et sigmoid :

Fonction d'activation	Accuracy	Taux d'erreur	Loss
ReLU	85,00 %	15,00 %	0,4795
Tanh	86,50 %	13,50 %	0,5492
Sigmoïde	82,50 %	17,50 %	0,9673

Tableau 8 - Comparaison des activations sur une couche cachée de 16 neurones

On peut constater que la fonction tanh obtient la meilleure accuracy (à 1,5% de plus que ReLU). Cependant, ReLU présente la plus faible perte, ce qui indique de meilleures performances. La fonction sigmoïde obtient les moins bons résultats sur les deux mesures.

ReLU et tanh semblent donc toutes les deux adaptées à notre réseau, avec un léger avantage pour tanh.

5. Réseau final

En combinant les recherches sur nos hyperparamètres, et en utilisant les mesures JCD, CEDD et FCTH, on obtient finalement un réseau à **une seule couche cachée** à **32 neurones**, utilisant la fonction d'activation **tanh**, entraîné sur **20 époques**.

Configuration	Accuracy	Taux d'erreur	Loss
Réseau dense retenu	87,00%	13,00%	0,4825

Tableau 9 - Résultats de la concaténation retenue

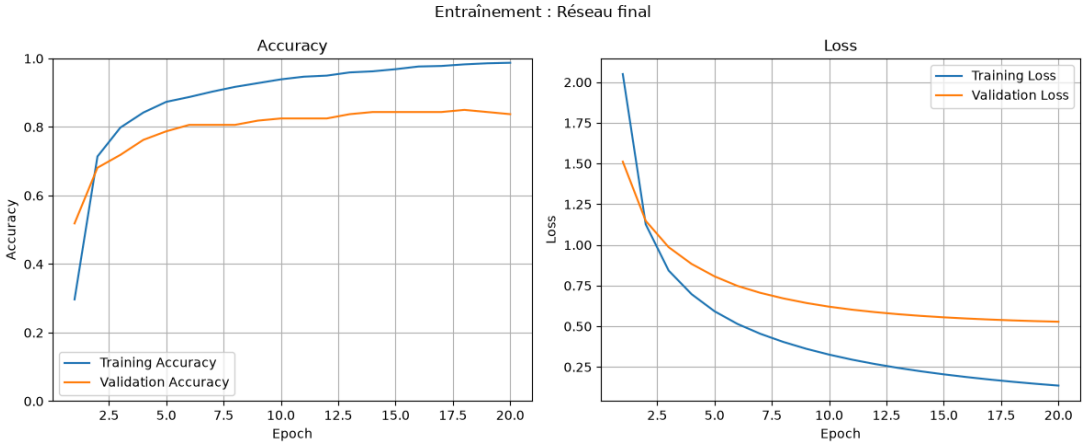


Figure 28 - Courbes du réseau dense retenu

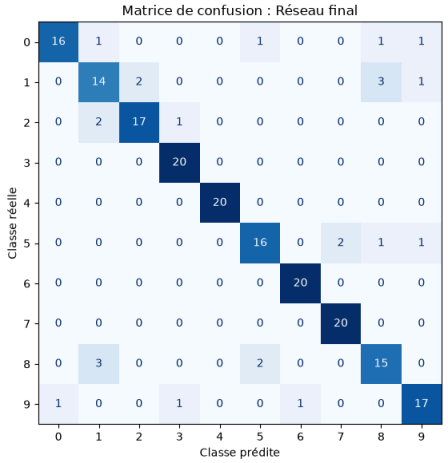


Figure 29 - Matrice de confusion du réseau dense retenu

Notre réseau final possède une accuracy de **87,00 %** et une perte de 0,4825, contre 79,00% et 0,6020 avant optimisation des hyperparamètres. On observe donc bien une nette amélioration de la précision de classement de notre modèle (gain de 8%).

Les problèmes de classification restants semblent majoritairement concerner des images des classes 1 et 8, souvent confondues l'une pour l'autre. Il s'agit d'une piste potentielle d'amélioration de notre modèle, peut-être à l'aide d'un plus grand jeu de données.

Comparaison avec l'algorithme KppV

1. **Réalisation de l'algorithme KppV**

On a, à présent, un réseau neuronal entraîné dont les hyperparamètres ont été optimisés. Il peut être intéressant de le comparer à un **algorithme KppV** :

```
# Dictionnaire des descripteurs pour passer tous les descripteurs
descripteurs = {
    "PHOG": X_PHOG,
    "JCD": X_JCD,
    "CEDD": X_CEDD,
    "FCTH": X_FCTH,
    "FuzzyColorHistogram": X_Fuzzy,
    "Concatenation": X_concat
}

# Séparation entre apprentissage et test
indices = np.arange(len(label))
idx_train, idx_test = train_test_split(
    indices,
    test_size=0.2,
    random_state=42,
    stratify=label
)

# pour stocker les résultats des descripteurs
resultats = {}
```

Extrait 7 - Mise en place du dictionnaire et des données d'entraînement et de test

Dans le code les classifications sont mises dans un dictionnaire pour pouvoir toutes les calculer d'un coup dans la suite du programme.

Pour l'entraînement, on garde la répartition 80/20 utilisée par Full Connect.

```
for nom, X in descripteurs.items():

    # Vérification que chaque image n'a qu'un label
    assert len(X) == len(label)

    #Caractéristiques des images d'apprentissage
    X_train = X[idx_train]
    #Caractéristiques des images de test
    X_test = X[idx_test]

    #Classes réelles des images d'apprentissage
    y_train = label[idx_train]
    #Classes réelles des images de test
    y_test = label[idx_test]

    # Pipeline : normalisation + KNN
    pipeline = Pipeline([
        # enchaîner plusieurs opérations dans un seul objet
        ("scaler", StandardScaler()),
        # Normaliser les caractéristiques numériques
        ("knn", KNeighborsClassifier())
        # l'algorithme KNN, les paramètres ne sont pas encore précisés
    ])
```

Extrait 8 - Suite du programme pour la mise en place de l'entraînement des données

Dans cette section nous entrons dans la boucle pour tester nos différentes classifications.
En premier lieu nous vérifions que chaque image a bien reçu un label pour éviter des erreurs futures, par la suite, nous récupérons les valeurs et classes des données d'entraînement et de test.

La pipeline est là pour exécuter la normalisation de nos valeurs avant de les envoyer directement dans l'algorithme de KNN.

```
    # Paramètres à tester pour rechercher le meilleur résultat
    parametres = {
        "knn__n_neighbors": [1, 3, 5, 7, 9, 11, 15],
        #on évite les nombres pairs pour la prise de décision
        "knn__weights": ["uniform", "distance"],
        "knn__metric": ["euclidean", "manhattan"]
    }

    recherche = GridSearchCV(
    # permet de rechercher automatiquement les meilleurs paramètres
    # d'un modèle en utilisant la validation croisée.
```

```
        pipeline,
        param_grid=parametres,
        cv=5,
        # cv : en combien de sous-ensembles on va diviser
        # nos images d'entraînement
        scoring="accuracy",
        n_jobs=-1
    )
```

Extrait 9 - Préparations des paramètres pour la fonction KNN et de GridSearchCV

Afin d'obtenir les résultats les plus précis avec la fonction KNN, nous utilisons plusieurs possibilités de paramètres :

- `knn__n_neighbors` : Le nombre de voisins à prendre en compte pour assigner une classe à la donnée étudiée. Nous avons le choix entre les nombres impairs entre 1 et 15.
- `knn__weights` : Il s'agit de la manière dont chaque voisin va impacter la décision. Dans "uniform" chaque voisin va impacter la décision de la même manière alors que dans "distance" les voisins les plus proches auront plus de poids que les voisins éloignés.
- `knn__metric` : Méthodes de calcul de distance entre les voisins. Nous avons la distance euclidienne et la distance de Manhattan.

À la suite de la mise en place des paramètres, nous utilisons `GridSearchCV` pour chercher le meilleur résultat de paramétrage parmi toutes les possibilités

```
# Entraînement
recherche.fit(X_train, y_train)

# Meilleur modèle
meilleur_modele = recherche.best_estimator_

# Utilisation sur les données de test
y_pred = meilleur_modele.predict(X_test)

# Évaluation du pourcentage de réussite
accuracy = accuracy_score(y_test, y_pred)

resultats[nom] = accuracy
```

Extrait 10 - Phase d'entraînement et d'évaluation

Pour finir nous réalisons l'entraînement suivi du test et de l'évaluation de la réussite du programme.

2. Résultats et comparaison

L'algorithme est testé sur les mêmes données que notre réseau : la concaténation de JCD, CEDD et FCTH.

Classe	précision	recall	f1-score	support
0	0.81	0.85	0.83	20
1	0.70	0.70	0.70	20
2	0.81	0.85	0.83	20
3	0.94	0.80	0.86	20
4	0.74	1.00	0.85	20
5	0.74	0.70	0.72	20
6	0.95	1.00	0.98	20
7	0.90	0.95	0.93	20
8	0.61	0.55	0.58	20
9	0.93	0.70	0.80	20

Tableau 10 - Mesures par classe du KppV sur les 200 images de test

`accuracy: 0.81`

On observe ainsi une accuracy de **81,00 %** pour l'algorithme KppV, contre 87,00% pour notre réseau neuronal. Sur les 200 images de test, cela représente 12 images supplémentaires correctement classées par le réseau.

On remarque également que les deux algorithmes rencontrent des difficultés sur les classes 1 et 8, malgré leurs méthodes de classification différentes. Cela pourrait signifier que les descripteurs utilisés ne permettent pas de distinguer suffisamment ces images.

En conclusion, selon ces observations, le réseau neuronal nous semble donc légèrement meilleur pour exploiter ces données que l'algorithme KppV.

V - Classification d'images par réseau profond : apprentissage des descripteurs

Dans cette partie, nous utilisons directement les images de la base Wang. Contrairement au perceptron multicouche, qui exploitait des descripteurs déjà calculés (PHOG, JCD, CEDD, FCTH et Fuzzy), le réseau apprend ici à extraire les caractéristiques utiles à la classification à l'aide de **couches de convolution**.

Les images sont redimensionnées à une taille commune de 256 x 256 pixels et représentées par leurs trois canaux de couleur. Les valeurs des pixels sont normalisées entre 0 et 1, et les étiquettes sont calculées à partir des noms de fichier, comme précédemment.

Nous conservons aussi la même séparation des données : 640 images pour l'entraînement, 160 pour la validation, et 200 pour le test. Les réseaux sont entraînés pendant 20 époques, avec des lots de 32 images, l'optimiseur Adam et la fonction de coût `categorical_crossentropy`.

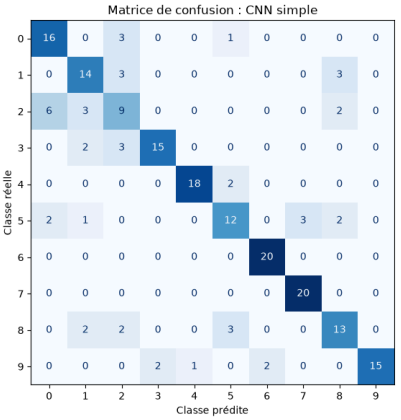
Architectures utilisées

On commence par un réseau simple, avec une convolution de 32 filtres de taille 3x3 avec une activation ReLU. S'ensuit un `MaxPooling` de taille 2 x 2, une couche flatten, le tout transmis à une couche de sortie Full Connected comme vu précédemment de 10 neurones utilisant `softmax`.

```
def create_simple_cnn(input_shape):
    # Convolution puis classification
    return keras.models.Sequential([
        layers.Input(shape=input_shape),
        layers.Conv2D(32, (3, 3), activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dense(num_classes, activation="softmax")
    ])
```

Extrait 11 - CNN Simple : extraction par convolution puis classification

A l'exécution, on observe sur ce réseau une accuracy de 76%, soit un taux d'erreur de 24% : près d'un quart de nos images sont mal classées.



A l'étude de notre entraînement, on constate assez vite l'origine du problème : l'accuracy sur l'ensemble d'entraînement plafonne dès la sixième époque alors qu'elle stagne sur la validation. Notre modèle s'est donc mal généralisé.

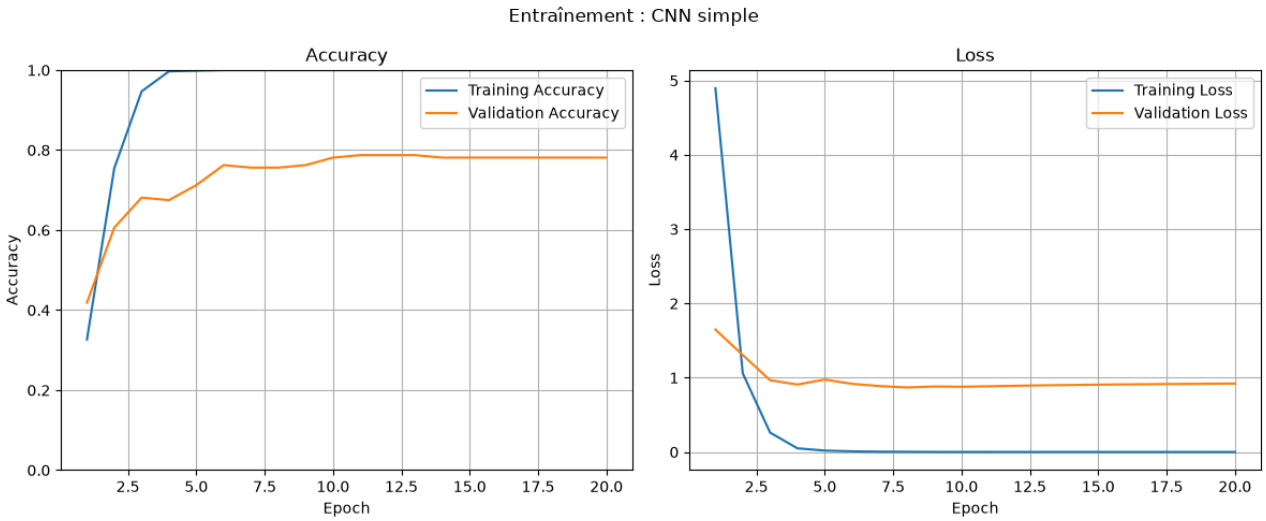


Figure 30 - Courbes du CNN simple entraîné pendant 20 époques

Une piste à suivre quant à la raison de ce surapprentissage est notre nombre de paramètres : le vecteur Flatten transmet 516 128 valeurs au classifieur, soit plus de 5 millions de paramètres à entraîner.

Notre deuxième architecture est plus profonde avec trois blocs convolution `MaxPooling`, avec 32, 64 et 128 filtres. La couche `Flatten` est remplacée par un `GlobalAveragePooling2D`, réduisant le nombre de valeurs transmises au classifieur (et donc le nombre de paramètres à apprendre).

La classification elle-même est réalisée par une couche dense de 128 neurones avec une activation ReLU, sortie de 10 avec softmax comme précédemment. Un dropout de 0,5 est ajouté pour tenter de limiter le surapprentissage.

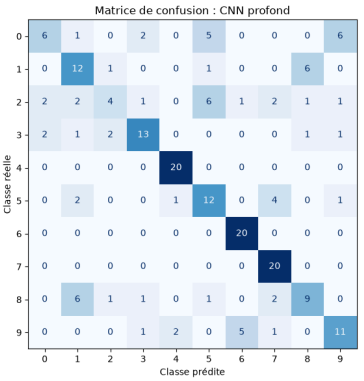
```
def create_deep_cnn(input_shape):
    model = keras.models.Sequential([
        layers.Input(shape=input_shape)
    ])

    for filters in (32, 64, 128):
        model.add(layers.Conv2D(
            filters, (3, 3), activation="relu", padding="same"
        ))
        model.add(layers.MaxPooling2D(pool_size=(2, 2)))

    model.add(layers.GlobalAveragePooling2D())
    model.add(layers.Dense(128, activation="relu"))
    model.add(layers.Dropout(0.5))
    model.add(layers.Dense(num_classes, activation="softmax"))
    return model
```

Extrait 12 - CNN plus profond

Malgré une réduction conséquente des paramètres à entraîner (de cinq millions à seulement 111050), on constate une **baisse de la précision** pour ce modèle plus profond : 63,5% d'accuracy (36,5% de taux d'erreur), soit une perte de précision de **12,5 points d'accuracy** par rapport au réseau simple.



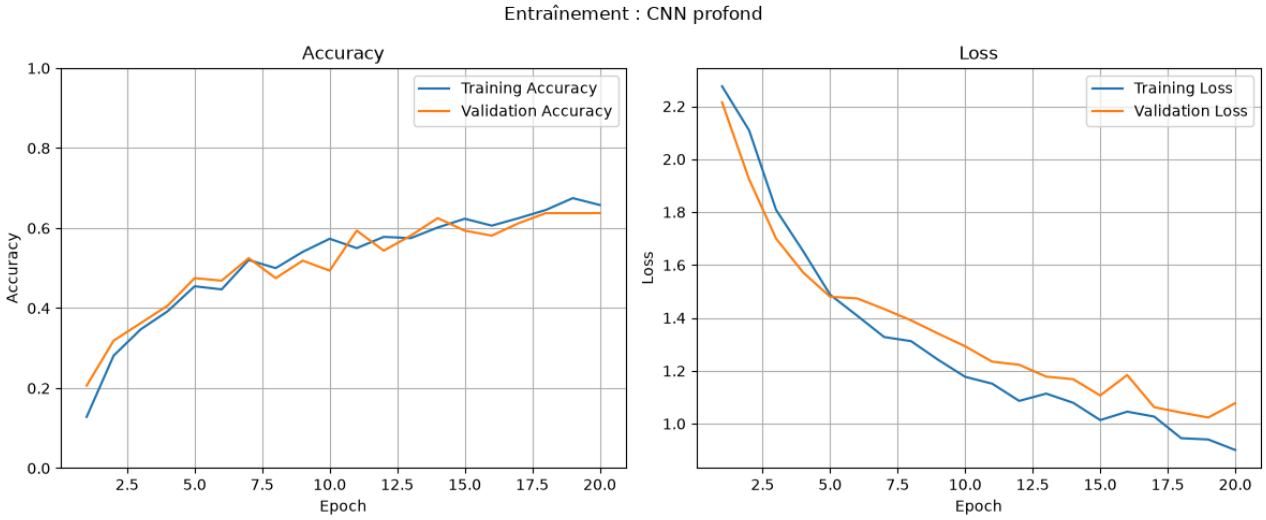


Figure 31 - Courbes du CNN profond après 20 époques

En regardant les courbes, le comportement de ce réseau semble bien différent de celui du premier modèle. Les pertes d'entraînement et de validation continuent globalement de diminuer, tandis que les deux courbes d'accuracy restent proches. On ne retrouve donc pas le fort écart entre apprentissage et validation observé avec le réseau simple !

A 20 époques, l'apprentissage semble encore en cours. Nous testons donc un nouvel entraînement sur la même architecture mais sur **50 époques**, auquel nous ajoutons une **augmentation des données**.

Cette augmentation applique des transformations aléatoires aux images : retournements horizontaux, rotations, zooms. L'objectif est de présenter au réseau différentes variations d'une même image pendant l'apprentissage tout en conservant sa classe.

On obtient alors une accuracy à 73% (taux d'erreur à 27%), avec une loss à 0,7547. On gagne ainsi 9,5 points d'accuracy par rapport au réseau précédent, soit 19 images supplémentaires correctement classées.

Cet entraînement améliore donc les résultats du premier essai, mais son accuracy demeure inférieure aux 76% obtenus sur le réseau simple.

Enfin, notre quatrième réseau teste une stratégie **d'apprentissage par transfert** avec **VGG16**, préentraîné sur ImageNet. Nous retirons sa classification d'origine et conservons ses couches d'extraction de caractéristiques.

Les poids de cet extracteur sont figés : seuls les paramètres des nouvelles couches de classification sont appris sur Wang. Nous ajoutons, comme précédemment : un GlobalAveragePooling, une couche dense de 128 neurones avec ReLU, un dropout de 0,5 et une sortie de dix neurones avec softmax.

```
def create_transfer_model(input_shape):
    from tensorflow.keras.applications import VGG16
    from tensorflow.keras.applications.vgg16 import preprocess_input

    base_model = VGG16(
        weights="imagenet", include_top=False, input_shape=input_shape
    )
    base_model.trainable = False

    return keras.models.Sequential([
        layers.Input(shape=input_shape),
        layers.Rescaling(255.0),
        layers.Lambda(preprocess_input),
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(128, activation="relu"),
        layers.Dropout(0.5),
        layers.Dense(num_classes, activation="softmax")
    ])
```

Extrait 13 - CNN avec extracteur VGG16

On obtient alors une classification presque parfaite : avec une **accuracy** de **94,5%** (taux d'erreur de 5,5%), seulement 11 images sur 200 sont mal classées ! Notre perte est seulement de 0,1823.

Les courbes d'entraînement montrent une progression rapide dès les premières époques, avec une accuracy d'apprentissage qui tend vers 100%. On constate un écart assez faible entre les ensembles d'entraînement et de validation, indiquant une généralisation bien plus satisfaisante que celle des modèles précédents.

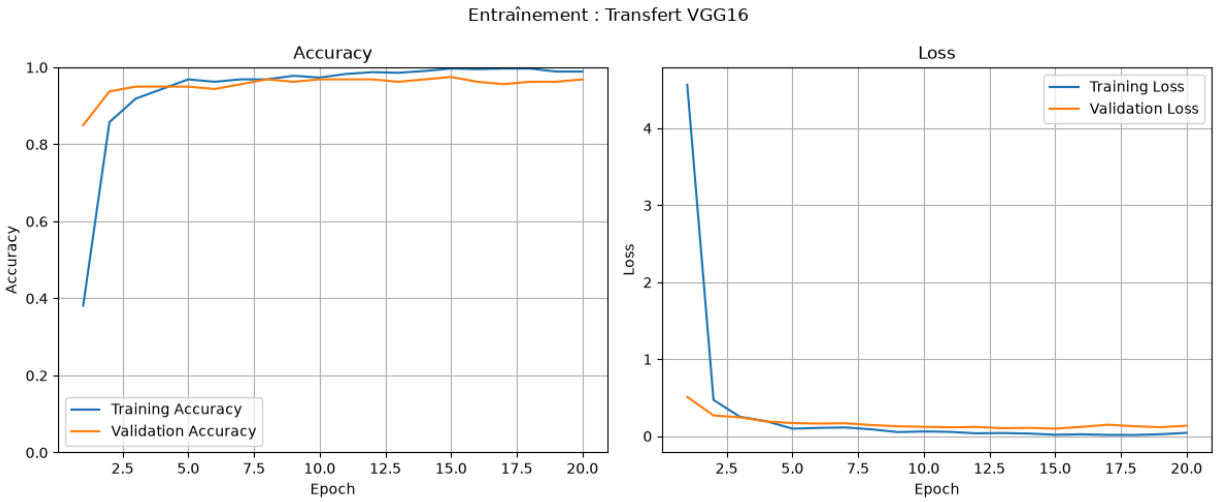


Figure 32 - Courbes de VGG16 sur 20 époques

En regardant la matrice de confusion, on peut préciser les erreurs restantes : les classes 3, 4, 5, 6, 7 et 9 sont parfaitement reconnues, tandis que les classes 1 et 2 ont le taux le plus bas de 16 images correctement reconnues sur 20.

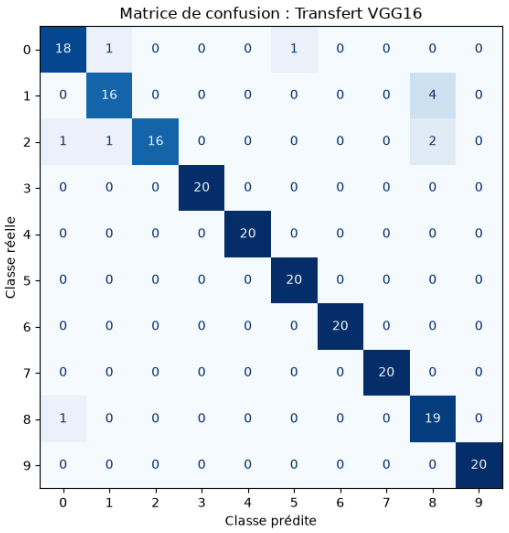


Figure 33 - Matrice de confusion du transfert VGG16

Enfin, pour comparer au perceptron multicouche final : nous avions une accuracy de 87% et une perte de 0,4825.

Avec VGG16, l'accuracy gagne 7,5 points en montant à 94,5%. Sur les 200 images de test, ce sont 15 images supplémentaires correctement classées. Le nombre d'erreurs passe de 26 à 11, la perte diminue de 0,4825 à 0,1823.

Ainsi, selon nos résultats, on peut conclure que les caractéristiques obtenues du réseau préentraîné permettent une **meilleure classification** que les descripteurs du perceptron multicouche. La confusion entre les classes 1 et 8, déjà observée précédemment, reste cependant présente et est une piste d'amélioration future pour notre réseau.